Final Project Report

Supervised Learning - 2024-2025

Project Topic: Analyze the environmental impact of industrial equipment and predict their CO2 emissions.

Student Name: [Sergio Fernandez Diz]

This project utilizes two datasets: **train_df**, which contains 5908 rows and 13 columns, and **test_df**, which has 1477 rows and 13 columns. There are two target variables: **CO2 Emission Level** and **CO2 Emission (g/hr)**. One will be used for classification, and the other for regression.

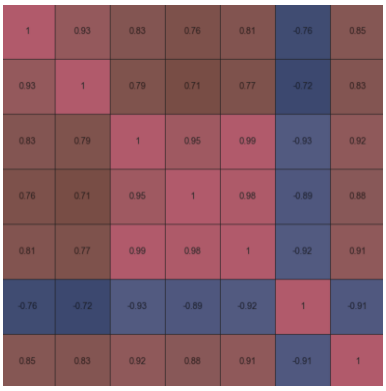
Processing data

a) Exploration Dataset: The dataset consists of 13 features and 5908 rows. By using the **.info()** method, it was found that the dataset does not contain any NaN values. However, there are several categorical features present [Manufacturer, Machine Model, Machinery Category, Transmission Type, Fuel Category, C02 Emission Level]. After checking the values of the categorical features, we noticed that some features have too many unique values to perform one-hot encoding effectively.

b) Categorical Features: . To address the issue of categorical features with too many unique values, we plotted the frequency of each unique value for every categorical feature. This allowed us to identify that certain features, such as **Manufacturer**, **Machine Model**, and **Transmission Type**, have values with too few samples, making it difficult for the model to generalize effectively. Additionally, the **Fuel Category** feature contains only four unique values, one of which, 'N', has only one sample.To resolve this, a threshold was introduced, and all values with insufficient representation were grouped under the category **"Others"**. For the value 'N' in **Fuel Category**, it was decided to remove it, as it does not provide useful information. The same features were grouped under **"Others"** in **test_df** as well, ensuring consistency between the features in both the training and test datasets. The last important point is that, upon analyzing the target, we noticed that the dataset is imbalanced. This is crucial because it indicates that accuracy should not be used to evaluate the models. Instead, it would be better to use the **F1 score**.

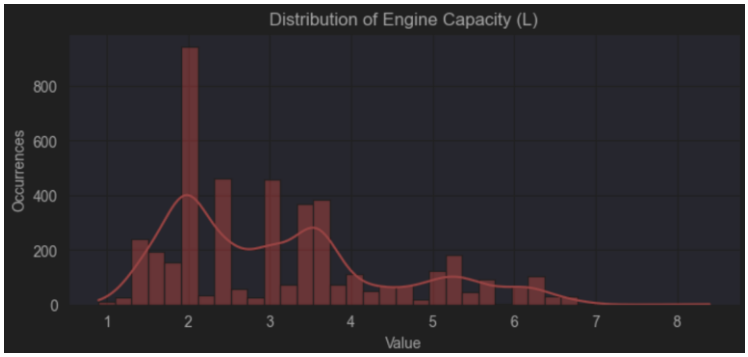
c) Find Duplicates: Using **.duplicated(),** a total of 1395 duplicate rows were found. To avoid biasing the models, these duplicates were **removed**, leaving a total of 4512 rows in the test dataset.

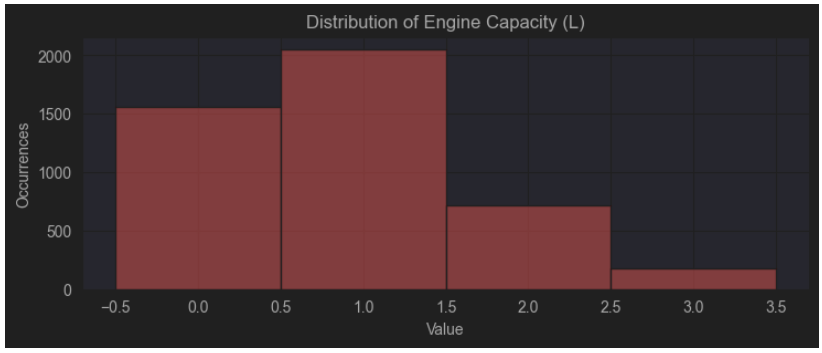
d) Outliers: For the outlier analysis, a boxplot was created, revealing the presence of outliers in several features. After analyzing the dataset and the meaning of the features, it was concluded that these outliers are plausible and represent car models with significantly better or worse values compared to the rest. To avoid losing valuable information, it was decided to retain these outliers.



e) Features Relationship: To analyze the correlations, a heatmap was used to show the relationships between the features. From the heatmap, we observed that the dataset contains many features that are strongly correlated with each other, and many represent the same concept but with slight variations. For example, **Fuel Consumption (Load)** is highly correlated with **Fuel Consumption (No-Load)**. Additionally, there are several other notable correlations. The heatmap suggests that, to simplify the model, we could remove many of these features. However, we decided to retain them for now. During the feature selection process, we will aim to keep only the most relevant features. By being aware of these correlations, we can simplify the model and improve its interpretability.
Moreover, scatterplots were created to analyze the correlation between the features and the target. The results indicate that features such as **Fuel Consumption** and **Average Fuel Efficiency** are highly important for the classification of the target.

f) Feauture Distribution: Through the distribution plots, we gathered several insights. Primarily, the numeric features exhibit a slightly **right-skewed distribution**, which is attributed to the presence of **outliers**. However, these outliers are reasonable and plausible. Additionally, we noticed that **Engine Cylinder** is a **numerical categorical** feature, with possible integer values ranging from 3 to 12. Regarding **Engine Capacity**, the feature displays significant **frequency peaks**, leading us to decide on applying **binning** to make the feature more interpretable for the model. No transformations, such as log or square root transformations, were applied to the skewed features, as they are not skewed enough to warrant such adjustments.





g) Features Encoding: For feature encoding, **one-hot encoding** was applied to the categorical features that do not follow a specific order. For the remaining categorical features, **ordinal encoding** was used. This results in a dataset with 135 features.

h) Split Dataset: Since the two targets, which will be used for two completely different tasks, are correlated with each other (one being continuous and the other categorical), both need to be removed. This is to avoid introducing information about the target during model training, which would lead to a **100% F1 score** due to the model effectively "cheating" by utilizing target-related information.

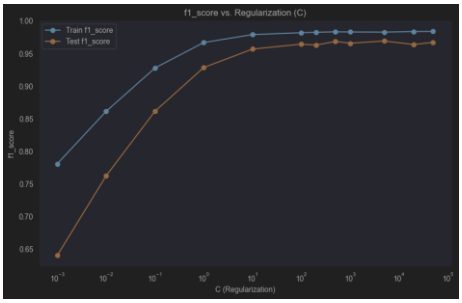
i) Feauture Selection: After encoding, we were left with a total of 135 features, which would make our models overly complex and vulnerable to issues like noise and overfitting. To make the models more interpretable and simpler, feature selection was applied. Three methods were used in total: **Univariate (SelectPercentile)** with a 30% threshold to significantly reduce complexity, **Iterative (RandomForest)**, which selected 30 features, and **Model-Based (Linear Regression)**. To combine the results from these three methods, we decided to keep only the features that appeared in at least two of the three models. In the end, the dataset that will be used consists of 26 features. This includes the features previously analyzed, which showed significant correlation with the target (such as **Average Fuel Consumption**, **Fuel Efficiency**, and others).

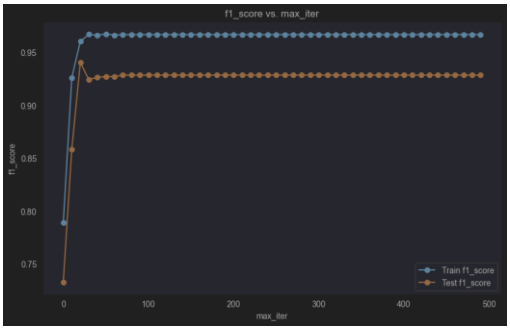
1. Analisys Models:

The project involves completing the entire feature processing pipeline to obtain a clean dataset for training and evaluating models. A total of 6 classification models and 1 regression model were built. For each model, hyperparameter tuning was performed based on the importance of the parameters, and the models were then evaluated on the test set.

(a) Softmax Regression

Since **Softmax Regression** is a model based on **gradient-based optimization** to minimize the loss function, varying scales of features can cause inefficiencies or lengthen the minimization process. With this in mind, the first step was to apply **RobustScaler**, as the dataset contains outliers. The **RobustScaler** scales the features based on the **median** and **IQR** (Interquartile Range) instead of the mean and standard deviation, making it more robust to outliers.Next, to perform hyperparameter tuning, two graphs were plotted to help determine the optimal values for each hyperparameter:





When choosing the value of C, we need to consider whether we want to maintain high regularization (low C) or reduce it to avoid potential complexity issues (high C). From the graph, we can observe that the optimal value for C is 10, which gives the best F1 score without significantly reducing the regularization. When choosing the value of C, we need to consider whether we want to maintain high regularization (low C) or reduce it to avoid potential complexity issues (high C). From the graph, we can observe that the optimal value for C is 10, which gives the best F1 score without significantly reducing the regularization. Regarding max_iter, to avoid unnecessarily increasing the number of iterations and achieving an insignificant improvement in the F1 score, we decided to stop at 100 iterations. This ensures that we obtain the expected results without increasing the model's complexity.

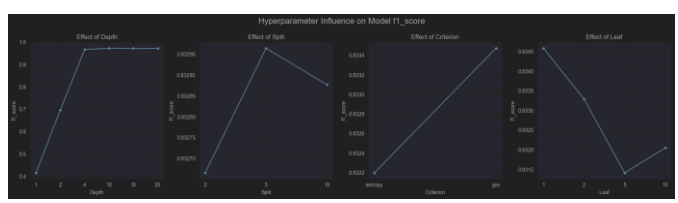
	precision	recall	f1-score	support
0	0.97	0.91	0.94	32
1	0.97	0.92	0.95	279
2	0.95	0.92	0.94	472
3	0.95	1.00	0.98	694
accuracy			0.96	1477
macro avg	0.96	0.94	0.95	1477
weighted avg	0.96	0.96	0.96	1477

The model demonstrates strong performance across all classes. The **F1 scores** are high for each class, with **Class 3** achieving the highest F1 score of **0.98** due to perfect recall. The **weighted average F1 score** is **0.96**, indicating overall good model performance. **Precision** and **recall** are also high, with the model achieving **0.96 accuracy** overall. However, there is some class imbalance, as **Class 3** has the largest number of samples.

(b) Decision Tree

First, we optimized the key hyperparameters: **max_depth**, which helps prevent the model from becoming too complex, and **min_samples_leaf**, which ensures that the leaves of the tree do not become too specific, thus reducing model complexity and mitigating noise. Additionally, **criterion** was considered to determine the best splitting criterion (e.g., Gini impurity or entropy).These hyperparameters were chosen to address the challenge of overfitting in a dataset with many features. By limiting the tree depth and controlling the minimum number of samples in each leaf, we prevent the model from fitting noise and becoming overly complex, improving its ability to generalize.

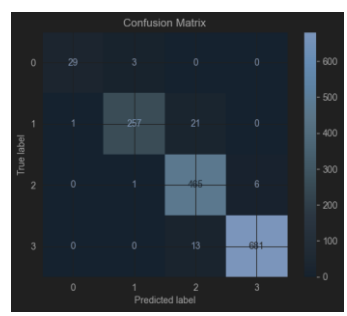
For **max_depth**, a value of **10** was chosen, as it allows for a high **F1 score** without significantly increasing the model complexity. For **min_samples_split**, we observed that it doesn't have as much of an impact as **max_depth**, but a value of **5** was selected to help ensure the model splits appropriately. Regarding **min_samples_leaf**, we noticed that starting with a higher value can help prevent the creation of overly specific leaves, which could lead to overfitting. A value of **5** was chosen to avoid this issue and ensure the model generalizes well.



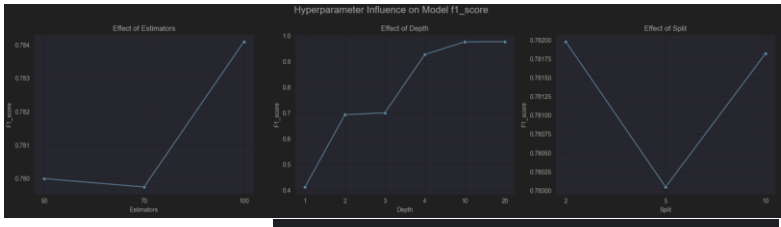
As we can see, the model performs very well, achieving slightly better results compared to the **Softmax** model. Regarding the **confusion matrix**, the main issues are **Class 1** being predicted as **Class 2** and **Class 3** being predicted as **Class 2**. Upon analyzing all the models, it becomes apparent that they all exhibit the same types of errors, which are likely caused by the complexity of the dataset. This complexity is consistent across all models, leading to similar misclassifications.

	precision	recall	f1-score	support
0	0.97	0.91	0.94	32
1	0.98	0.92	0.95	279
2	0.93	0.99	0.96	472
3	0.99	0.98	0.99	694
accuracy			0.97	1477
macro avg	0.97	0.95	0.96	1477
weighted avg	0.97	0.97	0.97	1477

F1 Score (weighted): 96.95%



(c) Random Forest



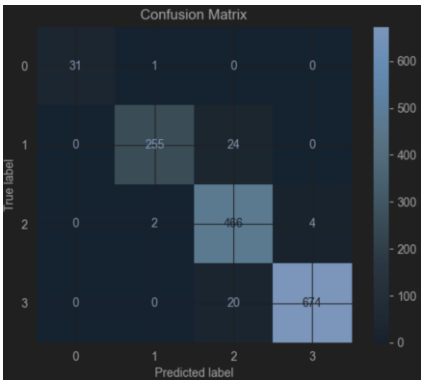
Similar to the process followed for the **Decision Tree**, the first step was to optimize the hyperparameters. As with the Decision Tree, to control the complexity, the **max_depth** and **min_samples_split** were optimized. Additionally, the **number of estimators** was also considered. After reviewing the graphs, the following values were selected:
50 for **n_estimators**, as increasing the number further did not provide significant advantages, thus keeping the model less complex. For **max_depth**, following the same reasoning as with the Decision Tree, a value of **10** was chosen. Finally, for **min_samples_split**, a value of **5** was selected.
As we can see, the model outperforms the previous ones, achieving very high values, even for the least represented class (Class 0). It reaches a mean accuracy of **0.98**, which is quite impressive. Regarding the **confusion matrix**, it follows the same pattern as the **Decision Tree**, with the difference being that the number of errors is lower.

	precision	recall	f1-score	support
0	1.00	0.91	0.95	32
1	0.97	0.95	0.96	279
2	0.96	0.97	0.97	472
3	0.99	1.00	0.99	694
accuracy			0.98	1477
macro avg	0.98	0.96	0.97	1477
weighted avg	0.98	0.98	0.98	1477

(d) AdaBoost

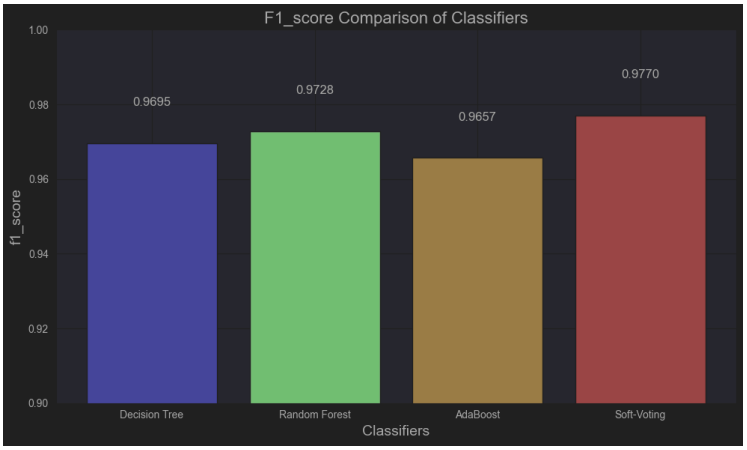
For the model, a **DecisionTree** was used as the weak learner in **AdaBoost**. The optimized parameters were **max_depth** to prevent overfitting, **n_estimators** to control complexity, and **learning_rate** to adjust the influence of each weak learner.In terms of performance, it is very similar to the **Decision Tree**. From the confusion matrix, we observe the same issues that are common across all models.

	precision	recall	f1-score	support
0	1.00	0.97	0.98	32
1	0.99	0.91	0.95	279
2	0.91	0.99	0.95	472
3	0.99	0.97	0.98	694
accuracy			0.97	1477
macro avg	0.97	0.96	0.97	1477
weighted avg	0.97	0.97	0.97	1477



(e) Soft Voting Assemble

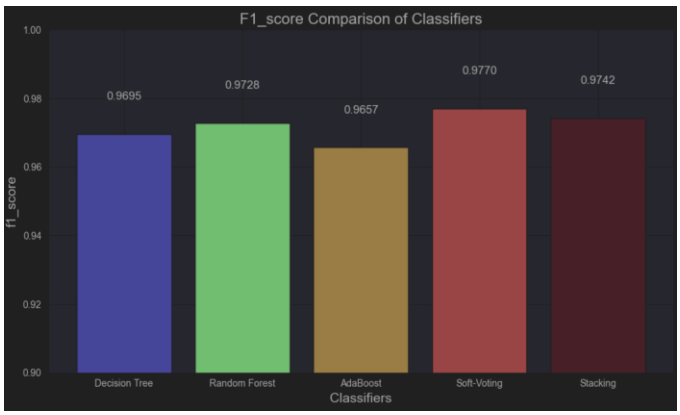
After building the previous classification models, a **Soft Voting** classifier was created. The models chosen for this ensemble are those that support this method (**Decision Tree**, **AdaBoost**, **Random Forest**), while **Softmax** was not used due to the need for scaling. After creating the ensemble, it was compared with the individual models.
Since we have an imbalanced dataset, the **F1 score** was used for comparison. As we can observe, the Soft Voting model performs better compared to the individual models, showing an improvement compared to singular models.



(f) Stacking Exploit

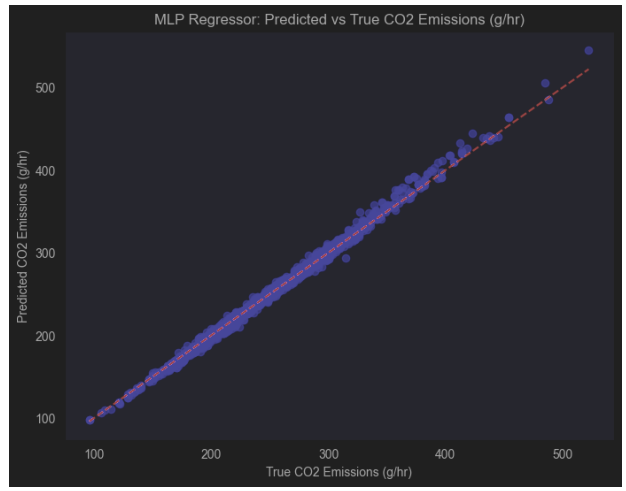
To implement **stacking**, the training data was first split into two parts. The first part was used to train the **base layer** models: **Decision Tree**, **Random Forest**, and **AdaBoost**. The second part was used to generate the **predictions** from these base models. These predictions were then combined to create a new feature set, which was used to train and optimize the **blender** model. The blender model chosen was **Logistic Regression**, and its hyperparameters, **C** and **max_iter**, were optimized using the same rationale as for **Softmax**.After training the blender, the **test dataset** was used to evaluate the performance of the stacking model, and its results were compared to those of the individual models.

The **stacking model** has shown **slightly better performance** compared to **other models**, but it still does not outperform **Soft Voting**, which remains the best model in terms of the **F1 score**. **Stacking** combines predictions from **Decision Tree**, **Random Forest**, and **AdaBoost**, which may lead to a slight improvement or average out the performance of the individual models. However, it does not bring significant additional value, as seen in its similar **F1 score** compared to **Soft Voting** and individual models.

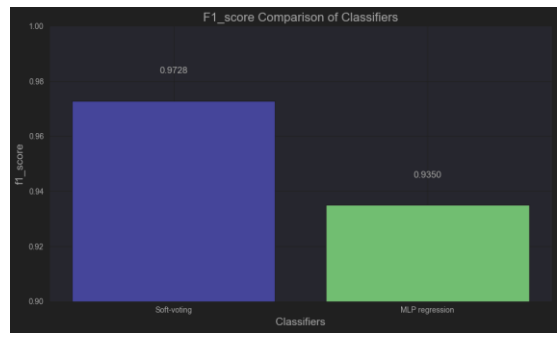


(g) MLP Regression

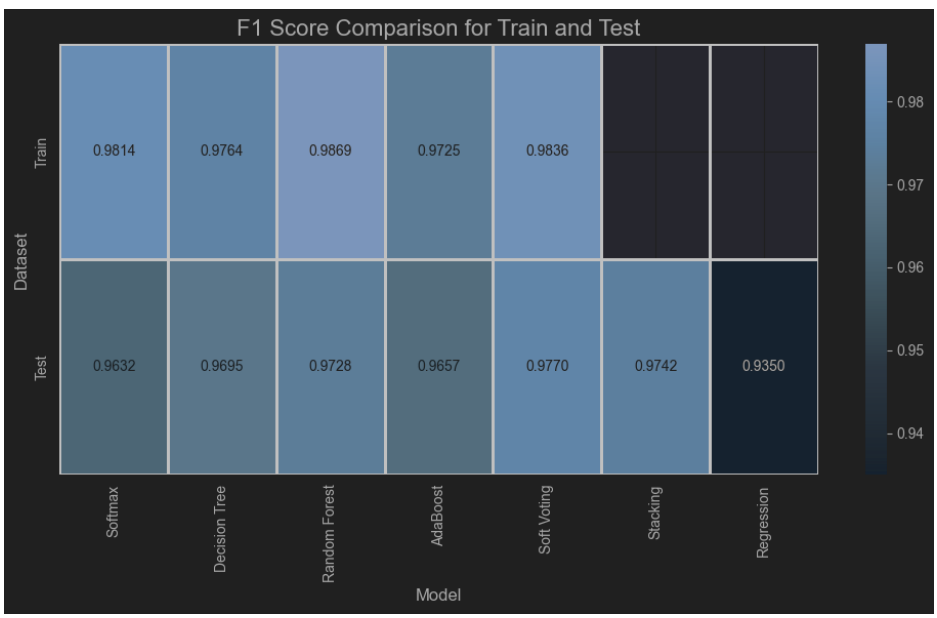
To predict the **numerical target**, an **MLP Regressor** model was implemented. The goal was to predict the continuous target value and then compare it with models that predict the same target in **categorical** form. To enable a fair comparison between regression and classification models, **binning** was applied to the regression predictions, transforming the continuous values into categorical classes for comparison with the classification models. The MLP Regressor model was defined and its hyperparameters **hidden_layer_sizes**, **activation**, **solver**, **learning_rate_init**, and **max_iter** were optimized using grid search. After selecting the best model, it was used to predict the target. The model's performance was evaluated using **RMSE** (4.51) and **R^2** (0.99). The low RMSE indicates the model's predictions are close to actual values, while the high R^2 shows that 99% of the target's variance is explained, meaning the model generalizes very well. At this stage, **binning** was applied to transform the continuous predictions into categorical classes, allowing for a direct comparison with classification models. This step makes the predictions compatible for evaluation with classification metrics.



The result is similar to the classification models, with slightly worse performance. The main issues identified in the confusion matrix follow the same pattern as the previous models.



(h) Final Comparison



As we can see, in general, the models achieved **high F1 scores**, which can be attributed to the dataset containing many features that ultimately represented the same underlying information. This redundancy made the task easier for the models and allowed us to reduce the complexity of the dataset, leading to simpler models that avoided overfitting and remained interpretable.

Upon comparing the different models, **Soft_Voting** emerged as the top performer. Following it are **Stacking**, **Random Forest**, and **Decision Tree**, which showed strong performance as well, but were less effective at capturing all the complexities of the data. On the other hand, **Softmax** and **Regression** performed the worst, likely due to the limitations of these models in capturing complex relationships between features and the target.One important observation is that the two ensemble methods (**Soft Voting** and **Stacking**) provide significant improvement over individual models.